MODULE 09 · BLOCK 2 - CONTROL FLOW - CONDITIONAL BLOCKS AND LOOPS

# Nested loops, break, continue and else

Loops inside loops, counting their passes, and exactly which loop break, continue and else affect.

BLOCK 2 of 4 EXAM WEIGHT 29% MODULE 09 of 28

admin@sthithapragna.com

The inner loop runs **all the way through** on every pass of the outer loop. break, continue and else only ever affect the **innermost** loop they belong to.

# The inner loop restarts on every outer pass

```
1  for i in range(1, 3):
2      for j in range(1, 4):
3          print(i, j, end="  ")
4      print()
```

2-3 For each value of i, j runs through 1, 2 and 3 from the beginning.

4 This print is inside the outer loop only: one new line per value of i.

Output: 1 1 1 2 1 3, then on the next line 2 1 2 2 2 3.

# The inner loop is the minute hand

| OUTER PASS | INNER PASSES | PRINTED |
| --- | --- | --- |
| `i = 1` | j = 1, 2, 3 | 1 1 1 2 1 3 |
| `i = 2` | j = 1, 2, 3 | 2 1 2 2 2 3 |
| `finished` | | |

THE PICTURE

The inner variable changes fastest. The outer variable only moves when the inner loop has finished.

# How many times does the inner body run?

| OUTER | INNER | INNER BODY RUNS |
|---|---|---|
| `range(3)` | range(4) | 3 x 4 = 12 |
| `range(4)` | range(i) | 0 + 1 + 2 + 3 = 6 |
| `range(1, 4)` | range(i) | 1 + 2 + 3 = 6 |
| `range(3)` | range(i, 3) | 3 + 2 + 1 = 6 |

EXAM TECHNIQUE

Fixed inner range: multiply. Inner range that uses the outer variable: add up each outer pass.

# When the inner range depends on the outer variable

```
1  for row in range(1, 4):
2      for col in range(row):
3          print("*", end="")
4      print()
```

2 Row 1 gets one star, row 2 gets two, row 3 gets three.

3-4 end="" keeps the stars on one line; the outer print() ends the row.

Output: *, then **, then ***, on three lines. print("*" * row) would do the same job.

# break ends the inner loop only

```
1  for i in range(3):
2      for j in range(3):
3          if j == 1:
4              break
5          print(i, j)
```

4 break ends the j loop. The i loop carries on with its next value.

Output: 0 0, then 1 0, then 2 0. The outer loop still makes all three passes.

# continue skips to the next pass of the inner loop

```
1  for i in range(2):
2      for j in range(3):
3          if j == 1:
4              continue
5          print(i, j, end="  ")
```

4 continue skips the rest of this inner pass. j moves on to 2.

Output: 0 0 0 2 1 0 1 2. Only the j == 1 pairs are missing.

# In the inner loop, or in the outer loop

BREAK INSIDE THE INNER LOOP

## Ends one inner loop

```
for i in range(3):
    for j in range(2):
        if i == 1:
            break
        print(i, j, end="  ")
```

0 0 0 1 2 0 2 1. When i is 1, the inner loop is cut short, then i moves on to 2.

BREAK IN THE OUTER LOOP

## Ends everything

```
for i in range(3):
    for j in range(2):
        print(i, j, end="  ")
    if i == 1:
        break
```

0 0 0 1 1 0 1 1. After the inner loop finishes for i == 1, the outer loop ends.

THE TIEBREAKER

Look at the indentation of the break. It ends whichever loop it is directly inside.

# Python has no break 2. Use a flag

```
1  found = False
2  for i in range(3):
3      for j in range(3):
4          if i * j == 2:
5              found = True
6              break
7      if found:
8          break
9  print(i, j)                 # 1 2
```

5-6 Record the discovery, then leave the inner loop.

7-8 Back in the outer loop, check the flag and leave that loop too.

# The inner else can run on every outer pass

```
1  for i in range(3):
2      for j in range(2):
3          if i == 1:
4              break
5      else:
6          print("no break at", i)
```

5 This else lines up with the inner for, so it belongs to the inner loop.

5-6 It runs each time the inner loop ends without break: for i = 0 and i = 2.

# What each keyword affects

| KEYWORD | EFFECT | WHICH LOOP |
|---|---|---|
| `break` | Ends the loop at once; its else is skipped | The innermost loop containing it |
| `continue` | Skips to the next pass | The innermost loop containing it |
| `else` | Runs if the loop ended without break | The loop it lines up with |
| `pass` | Does nothing | Not a loop keyword at all |

ON THE EXAM

None of these reaches past the innermost loop. To leave more than one, use a flag.

# Reusing the loop variable name

```
1  for i in range(2):
2      for i in range(3):
3          print(i, end=" ")
4  print("|", i)
```

2 The inner loop overwrites i, but the outer loop still gets its next value from its own range.

Output: 0 1 2 0 1 2 | 2. The outer loop still makes two passes.

# A while loop inside a for loop

```
1  for n in [3, 1]:
2      while n > 0:
3          print(n, end=" ")
4          n -= 1
5  print()                          # output: 3 2 1 1
```

1 The for loop hands the while loop a starting value each time.

2-4 The while loop counts that value down to zero, then the for loop moves on.

# What to carry into the exam

- The inner loop runs **completely** on every outer pass.
- Fixed inner range: multiply the counts. Inner range using the outer variable: add them.
- break and continue act on the **innermost** loop only. Use a flag to leave both.
- An else belongs to the loop it lines up with; an inner else can run on every outer pass.
- Reusing a loop variable name does not change how many passes the outer loop makes.

## How many stars does this program print?

```
for i in range(3):
    for j in range(i, 4):
        print("*")
```

A 9

B 12

C 6

D 7

◆ PAUSE AND DECIDE

## How many stars does this program print?

```
for i in range(3):
    for j in range(i, 4):
        print("*")
```

i = 0: 0 to 3, four stars. i = 1: three. i = 2: two. 4 + 3 + 2 = 9.

**A** 9

The inner range shrinks: 4 + 3 + 2.

**B** 12

*Multiplies 3 by 4. The inner range depends on i, so add instead.*

**C** 6

*Uses range(i, 3). The inner stop is 4.*

**D** 7

*Starts the inner loop at i + 1. range(i, 4) includes i.*

THE SIGNAL

If the inner range uses the outer variable, count each outer pass separately and add.

## What is the output of this program?

```
for i in range(1, 4):
    for j in range(1, 4):
        if j > i:
            break
        print(j, end="")
```

A 123123123

B 1

C 111222333

D 112123

◆ PAUSE AND DECIDE

## What is the output of this program?

```
for i in range(1, 4):
    for j in range(1, 4):
        if j > i:
            break
        print(j, end="")
```

i = 1 prints 1. i = 2 prints 12. i = 3 prints 123.

A 123123123

*Ignores the break. It cuts each row short.*

B 1

*Thinks break ends both loops. The outer loop carries on.*

C 111222333

*Prints i, not j.*

D 112123

Each row prints j up to i, then break ends that row only.

THE SIGNAL

A break in the inner loop shortens that pass of the inner loop. The outer loop is unaffected.

## What is the output of this program?

```
1  count = 0
2  for i in range(3):
3      for j in range(3):
4          if i == j:
5              continue
6          count += 1
7  print(count)
```

A 6

B 3

C 9

D 0

◆ PAUSE AND DECIDE

## What is the output of this program?

```
count = 0
for i in range(3):
    for j in range(3):
        if i == j:
            continue
        count += 1
print(count)
```

9 pairs in total. 3 have i == j: (0,0), (1,1), (2,2). 9 - 3 = 6.

**A** 6

All 9 pairs, minus the 3 where i equals j.

**B** 3

*Counts the skipped pairs instead of the counted ones.*

**C** 9

*Ignores the continue.*

**D** 0

*Treats continue as ending the loops. It skips one pass only.*

THE SIGNAL

Count the total passes, then subtract the ones continue skips.

## What is the output of this program?

```
for i in range(2):
    for j in range(2):
        if j == 2 * i:
            break
    else:
        print("X", end="")
    print(i, end="")
```

A X0X1

B 0X1

C 01

D X01

◆ PAUSE AND DECIDE

## What is the output of this program?

```
for i in range(2):
    for j in range(2):
        if j == 2 * i:
            break
    else:
        print("X", end="")
    print(i, end="")
```

i = 0: j == 0 breaks at once, no X, print 0. i = 1: j never equals 2, so X, then print 1.

A. X0X1

*Runs the else on both passes. For i = 0 the inner loop breaks.*

B. 0X1

Break on the first pass skips the else; no break on the second.

C. 01

*Never runs the else. For i = 1 the inner loop is not broken.*

D. X01

*Runs the else on the wrong pass.*

THE SIGNAL

An inner loop's else is decided afresh on every outer pass: was this inner loop broken?

## What is the output of this program?

```
found = False
for i in range(1, 4):
    for j in range(1, 4):
        if i + j == 4:
            found = True
            break
    if found:
        break
print(i, j)
```

A 3 1

B 2 2

C 1 3

D 3 3

◆ PAUSE AND DECIDE

## What is the output of this program?

```
found = False
for i in range(1, 4):
    for j in range(1, 4):
        if i + j == 4:
            found = True
            break
    if found:
        break
print(i, j)
```

A 3 1

*The last pair that sums to 4. The flag stops at the first.*

B 2 2

*Also sums to 4, but i = 1 comes first.*

C 1 3

The first pair found, in trace order. The flag ends both loops.

D 3 3

*Assumes both loops ran to the end. The flag stopped them.*

i = 1: j = 1, 2, 3. 1 + 3 == 4 at j = 3. Both loops end there.

THE SIGNAL

With a flag, both loops stop at the first match in trace order: outer value first, then inner.

## How many stars does this program print?

```
for i in range(4):
    if i % 2 == 1:
        continue
    for j in range(i):
        print("*", end="")
```

A Four

B Six

C None

D Two

◆ PAUSE AND DECIDE

## How many stars does this program print?

```
for i in range(4):
    if i % 2 == 1:
        continue
    for j in range(i):
        print("*", end="")
```

i = 0: range(0), no stars. i = 1: skipped. i = 2: two stars. i = 3: skipped.

**A** Four

*Counts i = 3 as well. Odd values are skipped.*

**B** Six

*Ignores the continue: 0 + 1 + 2 + 3.*

**C** None

*i = 2 is even, so its inner loop runs twice.*

**D** Two

Only even i reach the inner loop: 0 stars, then 2.

THE SIGNAL

A continue in the outer loop, before the inner loop, skips the whole inner loop for that pass.

## What is the output of this program?

```
for i in range(2):
    for i in range(3):
        pass
    print(i, end=" ")
```

A 0 1

B 2 2

C 2

D 1 2

◆ PAUSE AND DECIDE

## What is the output of this program?

```
for i in range(2):
    for i in range(3):
        pass
    print(i, end=" ")
```

The inner loop always leaves i at 2. The outer loop still makes two passes.

A `0 1`

*Forgets that the inner loop overwrites i before the print.*

B `2 2`

Two outer passes; each print follows an inner loop that ends at 2.

C `2`

*Thinks the overwritten i ends the outer loop. It takes its next value from range(2).*

D `1 2`

*Mixes the outer and inner values.*

THE SIGNAL

A for loop takes its next value from its own sequence, never from the current value of its variable.

> A break statement sits inside the inner loop of two nested loops. Both loops have an else clause.

**Which two statements are true when the break fires? (Choose two.)**

A. Both the inner and the outer loop end immediately

B. The else clauses of both loops are skipped

C. Only the innermost loop containing the break ends

D. The else clause of the loop that was broken is skipped

◆ PAUSE AND DECIDE

## Which two statements are true when the break fires? (Choose two.)

A Both the inner and the outer loop end immediately

*break reaches only the innermost loop. Use a flag to leave both.*

B The else clauses of both loops are skipped

*The outer loop was not broken, so its else can still run.*

C Only the innermost loop containing the break ends

The outer loop carries on with its next pass.

D The else clause of the loop that was broken is skipped

That is exactly what else means: no break happened.

THE SIGNAL

**break touches exactly one loop, and exactly one else: the innermost loop and its own else.**